

★ SPECIAL · PART 2

Building It With LangGraph

From the idea of an agent firm — to working code you can run

Part 1 asked WHAT an AI-native asset manager is and WHY it matters. Part 2 answers HOW: we build the agent firm step by step in LangGraph, the graph framework made for this exact job.

Why LangGraph Fits This Problem

An agent firm IS a graph — so use a graph framework

In Part 1 we drew the firm as a diagram: agents in boxes, arrows between them, a loop back from risk.

LangGraph lets you write that EXACT diagram as code — nodes, edges, and a shared state — then run it.

What it gives you

- A shared, persistent STATE across the whole workflow.
- Conditional routing and loops (not just straight lines).
- Built-in human-in-the-loop and memory.

Why it's trusted

- It's a state machine: explicit, traceable, debuggable.
- Runs in production at Uber, LinkedIn, Klarna, and more.
- Turns 'prompt spaghetti' into a clear, auditable graph.

Where We Are Going Today

Concepts, then a full build, then governance

- 1 The three LangGraph ideas**
State, nodes, and edges — that's the whole mental model. Plus reducers and conditional routing.
- 2 Build the agent firm, step by step**
State, the agent nodes, wiring the edges (with a risk loop), compile, and run.
- 3 Governance: human-in-the-loop**
Pause before big trades, get approval, resume — compiled into the graph, not bolted on.

By the end you could sketch — and code — the firm from Part 1 yourself.



1

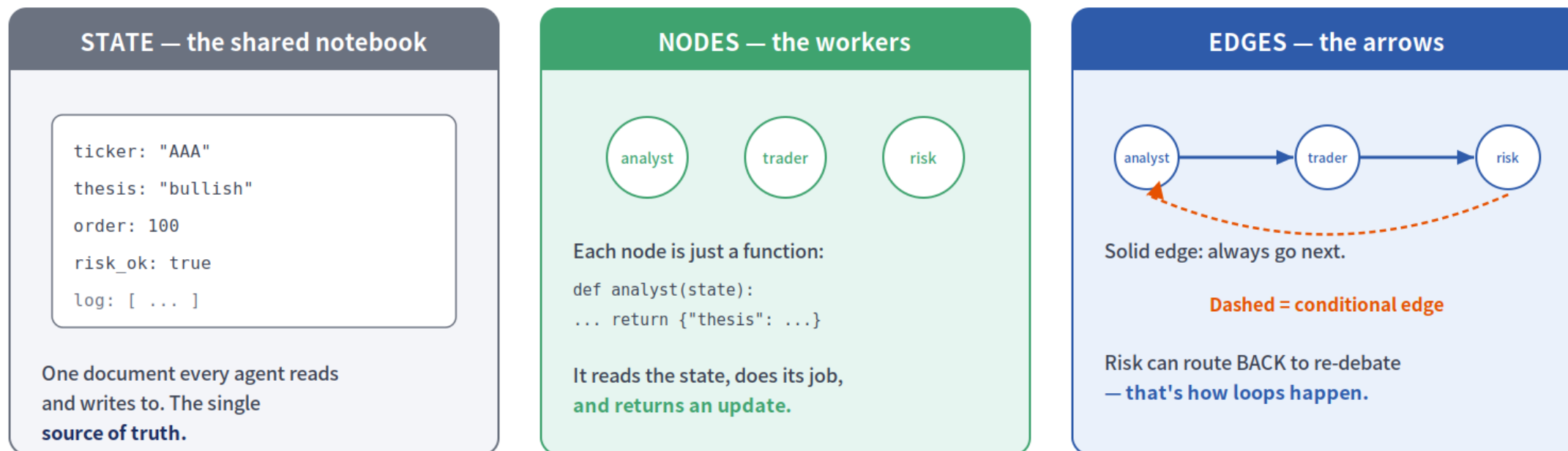
The Three Ideas

State, nodes, edges

LangGraph in Three Words

A shared notebook, worker functions, and arrows between them

LangGraph in Three Words — State, Nodes, Edges



That's the entire mental model. Everything else is detail on top of state, nodes, and edges.

Step 1 — Define the State

The shared notebook every agent reads and writes

What the state holds

- Every field the workflow needs to track.
- ticker, the thesis, the order, risk verdict, a log.
- Agents READ what they need and RETURN updates.
- A reducer says how to merge updates (overwrite or append).

firm.py • 1) the shared state

```
from typing import TypedDict, Annotated
import operator

class FirmState(TypedDict):
    ticker: str
    thesis: str          # filled by researcher
    order: int           # filled by trader
    risk_ok: bool        # filled by risk agent
    # 'log' APPENDS instead of overwriting:
    log: Annotated[list, operator.add]
```

Reducers — Overwrite or Append?

The one subtlety worth understanding up front

When two agents both write to the state, who wins? A reducer decides.

Default = overwrite (the latest value wins). But for a log, you want to APPEND every entry, not replace it.

Plain field (overwrite)

- order: int -> the newest value replaces the old.
- Good for single facts: the current order, the verdict.
- This is the default behavior.

Annotated field (append)

- log: Annotated[list, operator.add]
- Each agent's note is ADDED to the running list.
- This builds the audit trail we want (Week 14).

Step 2 — Write the Agent Nodes

Each agent is just a function: state in, update out

The node contract

- Takes the current state.
- Does its job (here, simple rules; real ones call an LLM).
- Returns a dict of fields to update.
- Same shape for every agent — easy to swap in an LLM.

firm.py • 2) the agent nodes

```
def analyst(state):
    view = 'bullish' if state['ticker'] else 'flat'
    return {'thesis': view, 'log': ['analyst: ' + view]}

def trader(state):
    size = 100 if state['thesis'] == 'bullish' else 0
    return {'order': size, 'log': ['trader: ' + str(size)]}

def risk(state):
    ok = abs(state['order']) <= 150          # a real limit
    return {'risk_ok': ok, 'log': ['risk: ' + str(ok)]}
```


2

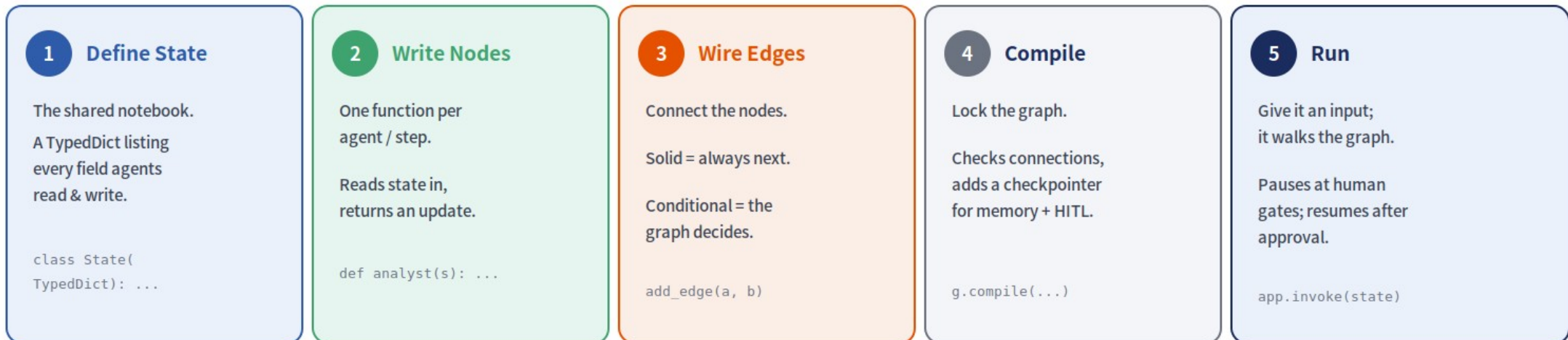
Wiring the Firm

Edges, loops, compile, run

Five Steps to Any LangGraph App

State, nodes, edges, compile, run — always the same shape

Five Steps to Build Any LangGraph App



We did State and Nodes in Section 1. Now: edges, compile, and run.

Step 3 — Wire the Edges

Connect the nodes into a workflow

Two kinds of edge

- Solid (add_edge): always go to the next node.
- Conditional: a function decides where to go.
- START and END are special built-in markers.
- Conditional edges are how you get loops & branches.

firm.py • 3) the edges

```
from langgraph.graph import StateGraph, START, END

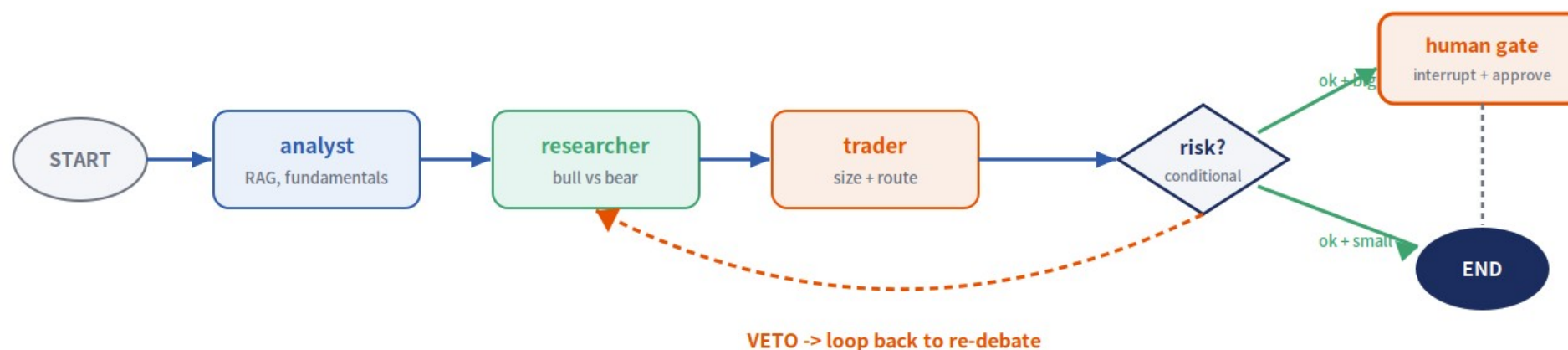
g = StateGraph(FirmState)
g.add_node('analyst', analyst)
g.add_node('trader', trader)
g.add_node('risk', risk)

g.add_edge(START, 'analyst') # always
g.add_edge('analyst', 'trader')
g.add_edge('trader', 'risk')
```

The Firm as a LangGraph

The Part-1 diagram, now an executable graph

The Firm as a LangGraph — One Picture, One Build



The picture IS the program.

In LangGraph you add these nodes and edges in code, compile, and run. The diagram and the implementation are the same thing.

The risk node's conditional edge creates the loop and the human gate — the picture is the program.

Step 3b — The Risk Loop (Conditional Edge)

Let the graph route based on the risk verdict

The routing rule

- If risk says NO -> go back to researcher (re-debate).
- If risk says YES -> proceed toward execution.
- A small function returns the NAME of the next node.
- This single idea gives you loops and vetoes.

firm.py • 3b) conditional routing

```
def route_after_risk(state):
    if not state['risk_ok']:
        return 'researcher'      # VETO -> loop back
    return END                   # approved -> done

g.add_conditional_edges(
    'risk',                      # from this node
    route_after_risk,            # the decision fn
    {'researcher': 'researcher', END: END},
)
```

Steps 4-5 — Compile and Run

Lock the graph, then give it an input

What compile does

- Validates that every edge connects real nodes.
- Catches cycles and dead ends early.
- Attaches a checkpointer (memory + human gates).
- After compile, the graph is locked and runnable.

firm.py • 4-5) compile and run

```
from langgraph.checkpoint.memory import MemorySaver

app = g.compile(checkpointer=MemorySaver())

start = {'ticker': 'AAA', 'log': []}
result = app.invoke(start,
                    config={'configurable': {'thread_id': 'demo-1'}})

print(result['order'], result['risk_ok'])
for line in result['log']: print(' ', line)
```

3

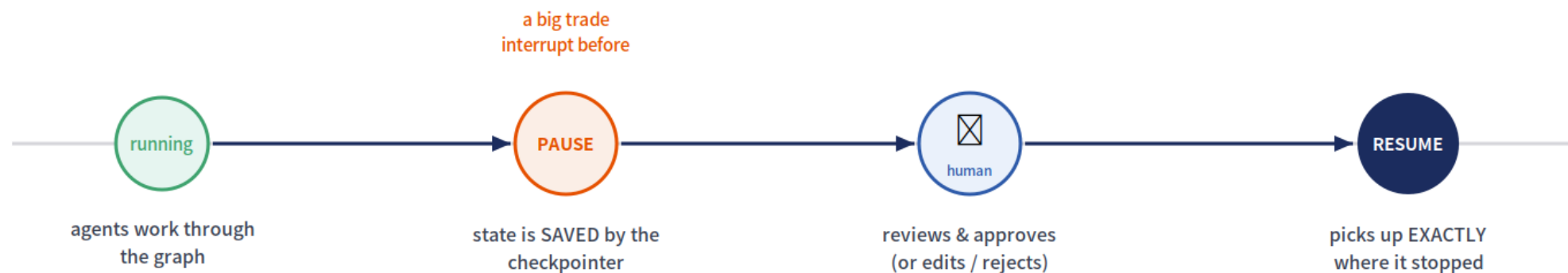
Governance in the Graph

Human-in-the-loop, the right way

Human-in-the-Loop — Pause, Approve, Resume

The checkpointer saves state so a run can stop and continue

Human-in-the-Loop — Pause, Approve, Resume



The human gate is COMPILED INTO the graph, not bolted on afterward

Because the checkpointer saves the full state, a pause can last seconds or days — then resume with no lost work. That's real governance.

The gate is compiled INTO the graph. A pause can last seconds or days, then resume with no lost work.

Adding a Human Gate

One line at compile time creates the pause

How it works

- `interrupt_before` pauses the graph before a node.
- The checkpointer saves the full state.
- A human reviews; you update state if needed.
- Resume with `invoke(None, ...)` — it picks up exactly there.

`firm.py` • human approval before execution

```
app = g.compile(  
    checkpointer=MemorySaver(),  
    interrupt_before=['execute'], # pause here  
)  
  
cfg = {'configurable': {'thread_id': 'demo-1'}}  
app.invoke(start, config=cfg) # runs, then PAUSES  
# ... human reviews the proposed trade ...  
app.invoke(None, config=cfg) # RESUME after approval
```

Autonomy by Risk Level, in the Graph

Small trades auto-run; big ones wait for a human

We don't pause for everything — that would defeat the point. We pause only where it matters.

A conditional edge sends small, low-risk trades straight to execution, and routes big ones through a human gate.

Low-risk path

- Order within limits and small -> auto-execute.
- The graph never pauses; agents finish the job.
- Fast where mistakes are cheap.

High-impact path

- Large or hard-to-undo -> route to the human gate.
- interrupt pauses; a person signs off.
- Then the same graph resumes and executes.

Seeing Inside the Firm

Every step is traceable — that's the governance win

The built-in audit trail

- Our 'log' field records every agent's action.
- The checkpointer stores every state transition.
- You can replay a run and see exactly what happened.
- Read WHY a trade was made (Week 14 explainability).

Tools that help

- LangSmith / Langfuse: trace latency, cost, and steps.
- Teams report much faster debugging with tracing on.
- Persistence: MemorySaver in dev, a database in prod.
- Observability is not optional for real money.

A graph you can trace, pause, and replay is a graph a risk committee can actually sign off on.



4

Your Turn

Run the firm and grow it

Two Common Multi-Agent Patterns

How to organize the graph as it grows

Supervisor

- One 'manager' node routes to specialist agents.
- The supervisor is an agent whose tools are other agents.
- Simple, central control — our firm's fund manager.
- Great starting point.

Hierarchical teams

- Each 'agent' is itself a whole sub-graph (a team).
- An analyst team, a risk team, each with members.
- Scales to bigger firms without one giant graph.
- Compose small graphs into larger ones.

Start with a supervisor; split into hierarchical teams only when one graph gets unwieldy.

Getting Started

Install LangGraph, or run the no-LLM toy version

Two ways to follow along

- Full: pip install langgraph, plug in any LLM.
- Toy: the nodes are plain functions (no API key).
- We use the toy version so it runs anywhere.
- The graph structure is identical either way.

What we'll do

- Lab 1: build & run the analyst-trader-risk graph.
- Lab 2: add the risk loop and a human gate.
- Watch the log to see the firm 'think'.
- Then swap a node's logic for a real LLM call.

`colab_setup.py` — **install (full) or skip (toy)**

```
# Full version (needs an API key for the LLM):  
# pip install langgraph langchain  
# Toy version below uses only the standard library —  
# same StateGraph shape, no model calls, runs anywhere.
```

Hands-On Lab 1 — Build & Run the Firm

State, nodes, edges, compile, invoke

Your task

- 1. Define the FirmState.
- 2. Add analyst, trader, risk nodes.
- 3. Wire START -> analyst -> trader -> risk -> END.
- 4. Compile and invoke.
- Question: what's in the log at the end?

colab_lab1.py • the StateGraph shape

```
from langgraph.graph import StateGraph, START, END
# (FirmState, analyst, trader, risk defined as on prior slides)

g = StateGraph(FirmState)
for name, fn in [('analyst',analyst),('trader',trader),('risk',risk)]:
    g.add_node(name, fn)
g.add_edge(START, 'analyst'); g.add_edge('analyst', 'trader')
g.add_edge('trader', 'risk'); g.add_edge('risk', END)

app = g.compile()
out = app.invoke({'ticker': 'AAA', 'log': []})
print('\n'.join(out['log']))
```

Hands-On Lab 2 — Risk Loop & Human Gate

Conditional routing and an approval pause

Your task

- 1. Replace risk -> END with a conditional edge.
- 2. On veto, route back to re-run analyst.
- 3. Compile with `interrupt_before` an 'execute' node.
- 4. Invoke, then resume after 'approval'.
- Question: which orders pause for a human?

colab_lab2.py • routing + interrupt

```
def route(state):
    return END if state['risk_ok'] else 'analyst' # veto loops
g.add_conditional_edges('risk', route,
                        {'analyst': 'analyst', END: END})

app = g.compile(checkpointer=MemorySaver(),
                interrupt_before=['execute'])
cfg = {'configurable': {'thread_id': 't1'}}
app.invoke({'ticker': 'AAA', 'log': []}, config=cfg) # pauses
app.invoke(None, config=cfg)                       # resumes
```


Comprehension Check

Try first, then reveal the answer below

Q1 In LangGraph, what is a 'node'?

- A A database row
- B A function (an agent or step) that reads state and returns an update ✓
- C A type of chart
- D The final answer

Answer: A node is a function — an agent or step — that reads the shared state and returns fields to update.

Q2 How do you create a loop or a risk veto in LangGraph?

- A You can't
- B With a conditional edge that routes back to an earlier node ✓
- C By deleting nodes
- D By running twice

Answer: A conditional edge returns the name of the next node, so it can route back to an earlier node — creating a loop.

Three Ideas, One Picture Each

1

State, nodes, edges

A shared notebook (state), worker functions (nodes), and arrows (edges). Conditional edges give loops and branches.

2

The diagram is the code

The Part-1 firm becomes a StateGraph: add nodes, wire edges, compile, invoke. The picture IS the program.

3

Governance is compiled in

A checkpointer plus `interrupt_before` gives pause-approve-resume. Loops, vetoes, and audit trails come for free.

LangGraph turns the abstract 'agent firm' of Part 1 into something you can build, run, and govern.

Part 2 — Three Things to Remember

1

Model the firm as a graph

State + nodes + edges. Each agent is a function; conditional edges create the risk loop and routing.

2

Five steps, every time

Define state, write nodes, wire edges, compile, run. The same recipe scales from a toy to production.

3

Governance lives in the graph

Checkpoint + interrupt = pause for human approval and resume. Traceable, replayable, auditable.

Part 1 = what & why; Part 2 = how. Together: a buildable, governable AI-native asset manager.

Try both labs, then replace the 'risk' node with a real LLM call and watch the same graph still work.

Taking It Further

From this toy graph to a real system

Make the agents real

- Swap each node's rules for an LLM with bound tools.
- Give the analyst RAG over filings (Week 12).
- Give the trader the execution model (Week 13).
- Add memory so agents recall past decisions.

Make it production-ready

- Swap MemorySaver for a database checkpointer.
- Add LangSmith/Langfuse tracing and evals.
- Set hard limits and kill-switches on tools.
- Keep the human gates on every high-impact action.

Same graph, bigger pieces. The structure you learned today is the structure real firms ship.